project-corebank 작업 정리

2026-11-13 · 정기예금(가입·중도해지·만기)과 상품 금리 변경 이력 / 리프레시 토큰·로그아웃(세션 즉시 무효화)

백엔드 **github.com/Csj930205/project-corebank** 화면 **github.com/Csj930205/project-corebank-front**

341	132	273 / 577	V19 + R
백엔드 테스트 **+36**	화면 테스트 **+15**	Postman 요청 / 검증 **+44 / +93**	DB 마이그레이션 **V18~V19**

증가 폭은 11-12 마감 기준(백엔드 305, 화면 117, Postman 229/484, V17). README 다음 계획 2·3순위. 1~5장은 정기예금, 6~8장은 리프레시 토큰.

1. 무엇을 만들었나

지금까지 상품은 "입출금 통장" 한 종류였고 금리는 바꿀 수 없었다. 실제 은행처럼 정기예금(기간·고정 금리·중도해지)을 넣고, 상품 금리를 바꿀 수 있게 하되 지난 이자는 흔들리지 않게 이력을 남겼다.

작업	내용
상품 종류	입출금(`DEMAND`)·정기예금(`TIME` , 가입 기간 1~60개월). JPA 상속 `SINGLE_TABLE` : `DepositProduct` (추상) → `DemandDepositProduct` , `TimeDepositProduct`
금리 변경·이력	`POST /admin/products/{id}/rate-changes` (운영), `deposit_product_rate` 에 "이 날짜부터 이 금리" 를 추가만. 입출금 적수는 이력에서 **그날 금리**를 찾는다
정기예금 가입	`POST /time-deposits` ★ 연결계좌에서 원금을 새 정기예금 계좌로 이체, 약정이율 = 가입 시점 상품 금리(고정), 만기일 = 가입일 + 기간
중도해지·만기	`POST /accounts/{id}/time-deposit/termination` ★, 매일 00:40 만기 처리(`POST /admin/time-deposits/maturities` 수동). 이자 → 원리금을 연결계좌로 이체 → 계좌 CLOSED
막기	정기예금 계좌의 입금·출금·이체·계좌 해지·정정 409 `TIME_DEPOSIT_RESTRICTED` , 가입 중 예금의 연결계좌 해지 409, 종류가 맞지 않는 상품 422
화면	입출금 계좌의 "정기예금" 탭(가입), 정기예금 계좌 화면(약정·진행 막대·만기 예상·오늘 해지 예상·중도해지), 관리자 상품 화면(종류·기간 등록, 금리 변경·이력, 정기예금 목록·만기 처리)

2. 돈의 흐름과 이자

단계	원장	계산
가입	연결계좌 TRANSFER_OUT → 정기예금 TRANSFER_IN (이체 기록 1건)	약정이율 복사, 만기일 = `plusMonths(기간)` (1/31 + 1개월 = 2/28)
해지	INTEREST(차 이자비용 / 대 예금 세후 + 예수금 세금) → 정기예금 TRANSFER_OUT → 연결계좌 TRANSFER_IN	적수 = 원금 × 이율 × 일수(가입일 ~ 해지일 전날). 세금·버림은 입출금과 같은 `InterestCalculator`

경과 기간	중도해지이율 (`EarlyTerminationRate`)	예: 12개월 3.65%, 100만 원
1개월 미만	연 0.10% (약정이 더 낮으면 약정)	당일 해지: 0일 → 이자 없이 원금만
3개월 미만	약정 × 30%	45일: 1.09% → 세전 1,343 / 세금 190 / 세후 1,153원

6개월 미만 / 그 뒤	약정 × 50% / × 70% (소수 둘째 자리 밑 버림)	만기(365일): 세전 36,500 / 세후 30,880원

3. 설계에서 고민한 것 (공부 포인트)

- **상속 전략 (학습 가이드 JPA 장):** 종류별로 다른 열이 `term_months` 하나라 `SINGLE_TABLE`. `JOINED` 는 계좌 목록마다 JOIN 이 하나 늘고 저장 때 INSERT 가 두 번, `TABLE_PER_CLASS` 는 계좌가 상품 하나를 외래키로 가리킬 수 없다. 대가(자식 열 NOT NULL 불가)는 CHECK 제약으로 메웠다.
- **지연 로딩 프록시와 `instanceof`:** `Account.product` 는 LAZY 라 처음엔 `DepositProduct` 를 상속한 프록시다. 실제로 정기예금이어도 `instanceof TimeDepositProduct` 가 **false**. 그래서 종류는 `isTimeDeposit()` 메서드로 묻는다 (호출은 프록시가 실제 객체에 넘긴다). 형변환이 필요하면 `Hibernate.unproxy`, JPQL 에서는 `TYPE(p) = TimeDepositProduct`. 테스트 `lazyProductProxyIsNotInstanceOfSubclass` 가 직접 보여 준다.
- **계좌를 상속하지 않고 1:1 로 붙임(`@MapsId`):** 정기예금도 잔액·원장·잠금·소유자 확인·해지는 입출금과 같다. 계좌는 그대로 두고 약정만 `time_deposit` 에 붙였다. 기본키가 곧 계좌 id 라 "계좌 하나에 계약 둘" 이 구조적으로 불가능하다.
- **1차 캐시가 만들 뻔한 두 번 지급:** 잠그기 전에 계약을 `findById` 로 읽으면 영속성 컨텍스트에 남아, 잠근 뒤 다시 `findById` 해도 DB 가 아니라 캐시의 옛 상태(ACTIVE)가 온다. 그 사이 다른 해지가 끝났다면 돈이 두 번 나간다. 그래서 잠금 전에는 연결계좌 번호만 값 조회(JPQL `SELECT t.payoutAccountId`)로 읽고, 계약 엔티티는 잠금 뒤에 처음 읽는다.
- **입출금을 어디서 막나:** `deposit()` / `withdraw()` 안에서 막으면 가입·이자·해지 때 잔액도 못 움직인다. 막을 것은 "누가 왜" 라서 고객 거래를 받는 서비스가 `checkDemandTransactionsAllowed()` 를 부른다(입금·출금·이체 양쪽·해지·정정 승인). 정정 요청 단계는 `TYPE()` 쿼리로 먼저 거른다.
- **금리 이력과 "그날 금리":** 적립이 상품의 지금 금리를 쓰면, 장애 뒤 몰아서 적립할 때 지난 날짜에 새 금리가 붙는다. 이력에서 `effective_from ≤ 그날` 중 가장 늦은 줄(`KEEP (DENSE_RANK LAST …)`)을 쓴다. 변경은 늘 오늘부터라 이미 쌓인 적수·지급한 이자와 어긋나지 않는다. 상품 행의 현재 금리는 동시 변경에 어긋나지 않게 상품 행을 잠그고 바꾼다.
- **재사용:** 원리금 이동은 새 거래 종류를 만들지 않고 기존 이체 기록·분개(`TransferService.recordInternal`)를 써서 거래내역에 상대 계좌가 보이고 일 마감 대사가 그대로 맞는다. 이자 지급 기록도 같은 `interest_payment` (계기 `MATURITY` · `EARLY_TERM`).
- **만기 처리는 배치가 아니라 반복문:** 하루 만기 건수는 적다. 건별 `TransactionTemplate` 으로 한 건 실패가 다른 고객 지급을 막지 않고, 실패 건은 ACTIVE 로 남아 다음 실행에서 다시 시도한다.

4. 검증

항목	결과	새로 확인한 것
백엔드 테스트 (Testcontainers Oracle)	329 통과	정기예금 통합 16개(시계를 옮기는 `MutableClock`): 가입 원금 이동·원장 합계, 같은 키 한 번만, 금리 바꿔도 약정이율 고정, 입출금·이체·해지·정정 거부, 45일 중도해지 금액, 당일 해지, 만기 전날은 대상 아님·만기 한 번만, 만기일 뒤 고객 해지 = 만기, 적립이 정기예금 건너뛰고 그날 금리 사용, 프록시 instanceof. 단위 8개(중도해지이율 경계, 이자 계산)
화면 테스트	127 통과	정기예금 상품 등록 요청 본문, 금리 변경, 만기 처리, 중도해지 두 번 눌러 확정·멱등성 키, 권한 없으면 안내
Postman 회귀 (실제 서버, 개발 DB)	261 / 552 통과	폴더 "16. 정기예금" 32요청: 재전송, 각종 거부 코드, 금리 이력, 당일 해지 후 연결계좌 해지 가능
실제 화면	확인	헤드리스 Chrome 으로 고객 가입 → 정기예금 가입 → 상세 화면, 관리자 상품 화면 캡처. 상품 표 열이 늘어 판매 버튼이 잘리는 것을 보고 종류를 이름 아래로 옮겼다

만들다 찾은 것

- Oracle `NUMBER` 는 끝자리 0 을 보관하지 않아 5.00 이 `BigDecimal` 5 로 읽힌다. 금리 비교는 문자열이 아니라 값(`isEqualByComparingTo`)으로.
- `Hibernate.unproxy(x, TimeDepositProduct.class)` 는 제네릭 타입이 맞지 않아 컴파일되지 않는다. `(TimeDepositProduct) Hibernate.unproxy(x)` .
- 기존 테스트(이자 통합)는 시계를 +40일로 옮겨 상품을 만든다. 금리 적용일을 앱 시계로 잡으면 "오늘" 적수에 금리가 없어진다. 적용일은 원장·개설 시각과 같은 서버 시각(`DbTime`)으로 맞췄다.

5. 정기예금의 남은 한계

알려진 한계

- 만기 후 이율·자동 재예치 없음. 늦게 처리돼도 이자는 만기일까지
- 중도해지이율 표가 코드에 하나. 상품별 표는 테이블 필요
- 금리 변경은 운영자 혼자(승인 없음), 늘 오늘부터. 예약 변경 없음
- 만기 처리는 서버 한 대 가정(동시 실행돼도 잠금 뒤 상태 확인으로 한 번만 지급)

6. 리프레시 토큰과 로그아웃 — 무엇을 만들었나

지금까지 JWT 는 서버가 저장하지 않아(무상태) 만료(30분) 전에는 끊을 방법이 없었다. 로그아웃은 화면이 토큰을 버리는 것뿐이었고, 관리자를 막아도 토큰은 30분 더 살았다. README 다음 계획 3순위.

작업	내용
로그인 세션 (V19)	로그인 한 번 = `auth_session` 한 줄(UUID). 액세스 토큰(15분)에 `sid` claim. JWT 디코더 검증기가 요청마다 세션이 살아 있는지 기본키로 확인 → 로그아웃·도난 탐지·계정 비활성이 **그 즉시** 401
리프레시 토큰	32바이트 무작위, DB 에는 SHA-256 해시만(`refresh_token`). `HttpOnly` · `Secure` · `SameSite=Strict` · `Path=/auth` 쿠키로만 준다. 쓰지 않고 30분, 로그인 뒤 최대 12시간
API	`POST /auth/refresh` (새 토큰 쌍, 회전), `POST /auth/logout` (세션 끊기, 늘 204). 실패는 401 `INVALID_REFRESH_TOKEN` / `REFRESH_TOKEN_REUSED` + 쿠키 삭제. 감사 기록 `TOKEN_REFRESH` · `LOGOUT`
화면	`api/client.ts` : 만료된 토큰은 요청 전에 리프레시, 401 이면 한 번 리프레시해 재전송, 동시 만료에도 리프레시 한 번(single-flight). 로그아웃 타이머는 리프레시 만료(30분 미사용) 기준. 로그아웃 화면이 서버에 세션 끊기 요청. Vite 프록시 `cookiePathRewrite`

7. 설계에서 고민한 것 (공부 포인트)

- **회전 + 재사용 탐지:** 리프레시할 때마다 토큰을 바꾼다. 훔친 토큰을 쓰는 순간 진짜 사용자와 도둑 중 한쪽은 "이미 쓴 토큰" 을 내민다 → 세션을 통째로 끊는다(도둑은 비밀번호를 모른다). 탭 두 개의 동시 리프레시는 10초 유예로 그 요청만 거부(정상 사용자가 갑자기 로그아웃되지 않게).
- **`noRollbackFor` (트랜잭션 장):** 재사용을 찾으면 세션을 끊고 예외를 던진다. 런타임 예외는 기본으로 롤백이라 "끊기" 도 사라진다. 이 예외만 롤백하지 않게 해 끊긴 기록이 커밋된다.
- **잠금:** 토큰 행을 `JOIN FETCH` 로 세션과 함께 `FOR UPDATE` (Oracle 은 OF 가 없으면 두 테이블 행 모두 잠금). 같은 토큰 동시 리프레시 5번 → 성공 1번, 나머지 유예 안 재사용.
- **`Persistable` (영속성 장):** 세션 id 를 코드에서 UUID 로 정하면 `save()` 가 "이미 있는 행" 으로 보고 merge(SELECT 먼저)를 한다. `isNew()` 로 새 객체라고 알려 바로 INSERT.
- **왜 bcrypt 가 아니라 SHA-256:** 토큰은 256비트 무작위라 사전 공격이 불가능하고, 해시로 바로 찾아야 한다(bcrypt 는 소금 때문에 값으로 찾을 수 없다). 비밀번호는 사람이 고른 값이라 bcrypt 다.
- **무상태를 일부 포기한 이유:** 요청마다 DB 조회가 하나 늘지만, 은행 업무에서는 "끊었는데 15분 더 쓸 수 있다" 가 더 큰 문제. 서명·만료·발급자를 먼저 검증해 위조 토큰 때문에 DB 를 보지 않는다.
- **리프레시 때 DB 재조회:** 역할·이름을 옛 토큰에서 옮기지 않고 DB 에서 다시 읽는다. 로그인할 수 없는 상태가 된 관리자는 그 때 세션이 끊긴다(다음 작업 "관리자 정지" 의 바탕).
- **화면은 요청이 있을 때만 리프레시:** 타이머로 미리 리프레시하면 화면만 열어 둬도 로그인이 끝없이 이어진다. "30분 미사용 로그아웃" 을 지키려고 요청 시점에만 한다.

8. 검증과 남은 한계

항목	결과	확인한 것
백엔드 테스트	341 통과	리프레시 12개(`MutableClock`): 쿠키 속성·해시 저장, 회전, 유예 뒤 재사용 → 세션 끊김·새 토큰까지 401, 유예 안 재사용은 세션 유지, 동시 5회 중 1번만, 로그아웃 즉시 401·재로그아웃 204, 30분 미사용, 12시간 상한(20분마다 35번 리프레시 뒤), 계정 비활성 → ACCOUNT_INACTIVE, 고객 리프레시, 감사 기록. 기존 테스트는 세션을 여는 발급으로 교체, sid 없는 옛 토큰은 401 로
화면 테스트	132 통과	만료 전 선제 리프레시, 401 재시도, 동시 만료 single-flight, 리프레시 401 이면 로그아웃, 리프레시 기한 기준 로그인 유지
Postman (실제 서버)	273 / 577 통과	폴더 "17. 리프레시 토큰·로그아웃" 12요청. 쿠키는 Set-Cookie 에서 꺼내 Cookie 헤더로 명시
실제 브라우저	확인	헤드리스 Chrome: 쿠키 HttpOnly·Secure·Strict· `/api/auth` , `document.cookie` 로 안 보임, 토큰 만료 흉내 → 자동 리프레시, 로그아웃 뒤 리프레시 401

알려진 한계

- 배포 직후 기존 토큰(sid 없음)은 모두 401 — 한 번 다시 로그인
- 요청마다 세션 DB 조회. 요청이 아주 많으면 캐시·Redis 필요
- 끝난 세션·쓴 토큰 정리 작업 없음
- 액세스 토큰은 sessionStorage(XSS 노출) — 15분·즉시 끊기로 피해만 줄인다
- 내 로그인 기기 목록·다른 기기 로그아웃 없음

다음 후보 (README 순위)

- 4. 관리자 계정 정지·비밀번호 변경(요청·승인, 정지 시 세션 모두 끊기)
- 5. ShedLock, 감사 로그 outbox, 부하 테스트